

CO1-AT1-Coding Challenge

← Back

QUESTIONS

Q1

Library Book Management
using Classes & Encapsulation a list
50 marks

Q2

University Admission System
using Method Overloading
50 marks

2/2 answered

Q1 Library Book Management using Classes & Encapsulation a list

50 marks

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

Your Code

```
1 import java.util.Scanner;  
2 public class Book{  
3     private int id;  
4     private String title,author;  
5     private double price;  
6     private boolean issued;  
7     Book(int id,String title,String author,double price){  
8         this.id=id;  
9         this.title=title;  
10        this.author=author;  
11        this.price=price;  
12        issued=false;  
13    }  
14    int getId(){  
15        return id;  
16    }  
17    void issueBook(){  
18        if(!issued){  
19            issued=true;  
20            System.out.println("Book Issued");  
21        }else{  
22            System.out.println("Already Issued");  
23        }  
24    }  
25    void returnBook(){
```

Input

```
1  
101  
Java  
James
```

Output

```
1.Add 2.Display 3.Search 4.Issue  
5.Return 6.Exit  
Book Id: Title: Author: Price:  
Book Added Successfully
```

CO1-AT1-Coding Challenge

← Back

QUESTIONS

Q1

Library Book Management
using Classes & Encapsulation a list
50 marks

Q2

University Admission System
using Method Overloading
50 marks

2/2 answered

Q2 University Admission System using Method Overloading

50 marks

Create an Admission class that overloads the method calculateFee() for:

- ? Undergraduate students,
- ? Postgraduate students,
- ? Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

Your Code

```
1 import java.util.Scanner;
2 public class Admission{
3     public double calculateFee(double baseFee,int credits,double feePerCredit){
4         return baseFee+(credits*feePerCredit);
5     }
6     public double calculateFee(double baseFee,double labFee){
7         return baseFee+labFee;
8     }
9     public double calculateFee(double baseFee,double discountPercentage){
10         if(isScholarship){
11             return baseFee-(baseFee*discountPercentage/100.0);
12         }
13         return baseFee;
14     }
15 public static void main(String[]args){
16     Scanner sc=new Scanner(System.in);
17     Admission obj=new Admission();
18     if(sc.hasNextInt()){
19         int studentType=sc.nextInt();
20         switch(studentType){
21             case 1:
22                 double ugBase=sc.nextDouble();
23                 int credits=sc.nextInt();
```

Input

```
1
50000
15
500
```

Output

57500.0